INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

Department of Information Technology

PROJECT REPORT

Event Management System using MVC Architecture (Java Swing GUI)

Course Name: Object Oriented Programming / Java Programming Lab

Name: Debdatta Roychoudhury

Enrollment Number: 12024052004042

Roll Number: 44

Section: A

Semester: 4th

Instructor Name: D.Samanta, M.Sharma

Certificate

This is to certify that Debdatta Roychoudhury (Roll No. 44), Enrollment No. 12024052004042, a student of Section – A from the Department of Information Technology, Institute of Engineering and Management, Kolkata, has successfully completed the project titled "Event Management System using MVC Architecture (Java Swing GUI)" during the academic year 2025–26 under the supervision and guidance of the department.

The project work presented in this report is the original work of the student and has not been submitted elsewhere for the award of any degree, diploma, or certificate.

Instructor Signature _____ Date _____

Place: Kolkata

2. Problem Statement

Managing event details manually can be difficult, especially when multiple events need to be tracked, updated, or removed over time. Traditional methods such as notebooks or spreadsheets are often time-consuming, error-prone, and difficult to maintain. As the number of events increases, it becomes harder to keep records organized and consistent.

To solve this problem, this project presents a desktop-based Event Management System built using Java Swing GUI and designed with the MVC (Model-View-Controller) architecture. The system allows users to perform the essential CRUD operations: Add, View, Update, and Delete event records. The application provides a simple graphical interface where users can enter event information such as event ID, event name, date, venue, and organizer.

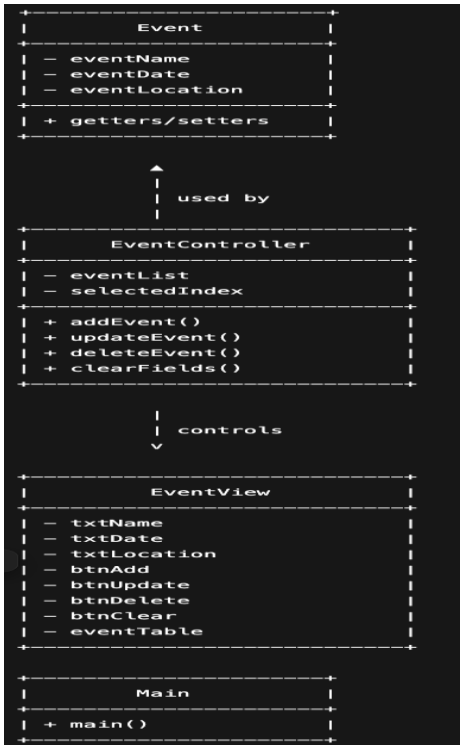
The main goal of this project is to create a beginner-friendly yet well-structured software system that demonstrates Java GUI development, object-oriented programming, and clean software architecture in a practical and understandable way.

3. System Design

3.1 Overview of the System

The Event Management System is a desktop GUI application that enables users to manage event records in an organized way. The project follows the MVC architecture, which separates the Model (data), View (GUI), and Controller (logic). This separation improves readability, maintenance, and modularity.

3.2 UML Class Diagram (Mandatory)



3.3 Description of Major Classes

1. Event.java (Model Class): Stores event data such as event ID, event name, date, venue, and organizer.

2. EventManagementGUI.java (View Class): Builds the graphical user interface using Java Swing components.

3. EventController.java (Controller Class): Handles user actions like Add, Update, Delete, and Clear.

4. Main.java: Entry point of the application.

3.4 Package Structure

```
EventManagementSystem/
└── src/
    ├── model/
    │   └── Event.java
    ├── view/
    │   └── EventManagementGUI.java
    ├── controller/
    │   └── EventController.java
    └── Main.java
```

3.5 Interaction Flow

- User opens the application.
- GUI window is displayed.
- User enters event details in the form.
- User clicks Add.
- Controller reads data from the GUI.
- Controller creates an Event object.
- Event is stored in an ArrayList.
- Table is refreshed and updated.
- If a row is selected, the user can Update or Delete it.

4. OOP Concepts Demonstrated

4.1 Abstraction

Where it is used: In the Event model class, event data is represented through a dedicated class.

Why it is used: To support clean, maintainable, and object-oriented design.

```
public class Event {
    private String eventId;
    private String eventName;
    private String date;
    private String venue;
    private String organizer;
}
```

Explanation: This groups related event data into one meaningful object.

4.2 Encapsulation

Where it is used: Private fields and public getter/setter methods in Event.java.

Why it is used: To support clean, maintainable, and object-oriented design.

```
private String eventName;

public String getEventName() {
    return eventName;
}

public void setEventName(String eventName) {
    this.eventName = eventName;
}
```

Explanation: The variable is private and accessed in a controlled way.

4.3 Inheritance

Where it is used: The GUI class extends JFrame.

Why it is used: To support clean, maintainable, and object-oriented design.

```
public class EventManagementGUI extends JFrame {
    public EventManagementGUI() {
        setTitle("Event Management System");
        setSize(800, 500);
    }
}
```

Explanation: By extending JFrame, the class becomes a full desktop window.

4.4 Polymorphism

Where it is used: Action listeners and overridden methods in Swing event handling.

Why it is used: To support clean, maintainable, and object-oriented design.

```
addButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        addEvent();
    }
});
```

Explanation: The actionPerformed() method is overridden to define custom behavior.

4.5 Exception Handling

Where it is used: Selection validation during Update/Delete operations.

Why it is used: To support clean, maintainable, and object-oriented design.

```
if (selectedRow == -1) {
    JOptionPane.showMessageDialog(gui, "Please select a row first.");
    return;
}
```

Explanation: This prevents errors when no row is selected.

4.6 Collections

Where it is used: ArrayList<Event> stores all event records.

Why it is used: To support clean, maintainable, and object-oriented design.

```
private ArrayList<Event> eventList = new ArrayList<>();

public void addEvent(Event event) {
    eventList.add(event);
}
```

Explanation: ArrayList allows dynamic storage of multiple events.

5. Implementation Highlights

5.1 Adding a New Event

```
String id = gui.eventIdField.getText();
String name = gui.eventNameField.getText();
Event event = new Event(id, name, date, venue, organizer);
eventList.add(event);
refreshTable();
```

Justification: This is the core CRUD functionality where user input is converted into an object and stored.

5.2 Updating Selected Event

```
int row = gui.eventTable.getSelectedRow();
Event event = eventList.get(row);
event.setEventName(gui.eventNameField.getText());
```

```
event.setDate(gui.dateField.getText());
refreshTable();
```

Justification: This demonstrates modification of an existing record.

5.3 Deleting an Event

```
int row = gui.eventTable.getSelectedRow();
if (row != -1) {
    eventList.remove(row);
    refreshTable();
}
```

Justification: This removes a selected event safely from the list.

6. Testing & Error Handling

Test cases considered:

Test Case ID	Scenario	Expected Result
TC1	Add valid event details	Event should be added to table
TC2	Update selected row	Existing event should be modified
TC3	Delete selected row	Event should be removed
TC4	Click Update without selection	Warning message should appear
TC5	Click Delete without selection	Warning message should appear

Edge cases handled: Update/Delete without selection, clearing partial input, and basic validation checks.

Failure scenarios: Empty fields, no row selected, and invalid data format are handled using warning dialogs and validation.

7. Git Workflow

Insert screenshot of commit history here (minimum 10 meaningful commits).

Development progression: project structure creation, model class, GUI layout, table integration, Add feature, row selection, Update feature, Delete feature, Clear feature, UML diagrams, **README**, and final testing.

8. Conclusion & Future Scope

8.1 Conclusion

The Event Management System using MVC Architecture (Java Swing GUI) is a simple yet effective desktop application for managing event records. It demonstrates Java Swing GUI, CRUD operations, MVC design, and OOP principles in a practical and beginner-friendly way.

8.2 Future Scope

- Database support using MySQL or SQLite
- Search and filter functionality
- Login and authentication system
- Export to PDF or Excel
- Better input validation and improved UI design

Appendix

Appendix A: Initial Project Proposal

Click link

Appendix B: Review Mail Received (APPROVED_MINOR)

PCCCS495 Proposal Review – APPROVE...

S soumadip.biswas@ieminternational.ai
soumadip.biswas@ieminternational.ai

To You Debdatta.Roychoudhury2024@iem.edu.in
Tuesday 10 March at 10:01 am

Reply-To: soumadip.biswas@ieminternational.ai

Dear Debdatta Roychoudhury ,

Project: Student Management System using MVC Architecture in Java
Decision: APPROVED_MINOR
Score: 88

Strengths:
The proposal is clear, well-structured, and appropriately scoped for a university Java OOP project. It clearly identifies a relevant problem and outlines a practical solution using the MVC architectural pattern. All core OOP concepts (Abstraction, Encapsulation, Exception Handling, Collections) are explicitly listed as being addressed, and the separation of concerns within the MVC design is well-articulated. The features are realistic and provide ample opportunity to demonstrate these concepts.